

CAST: Enhancing Code Summarization with Hierarchical Splitting and Reconstruction of Abstract Syntax Trees

NLP팀 박희수, **신동진**

추상 구문 트리의 계층적 분할 및 재구성을 통한 코드 요약 강화

Task: Code Summarization

- 소스 코드를 간결한 자연어로 나타내는 task
- 소프트웨어 유지보수와 프로그램의 이해에 중요한 역할
- 하지만 노동과 시간이 많이 듦 개발자가 코드에 대한 좋은 요약을 직접 작성해야 함
- Methods
 - 전통적: 복잡한 문법이나 구조는 무시하고 소스 코드를 일반 텍스트처럼 간주하고 룰 기반 혹은 IR 기반으로 접근
 - 최근: ASTs(abstract syntax trees)를 도입
추상구문

Limitations

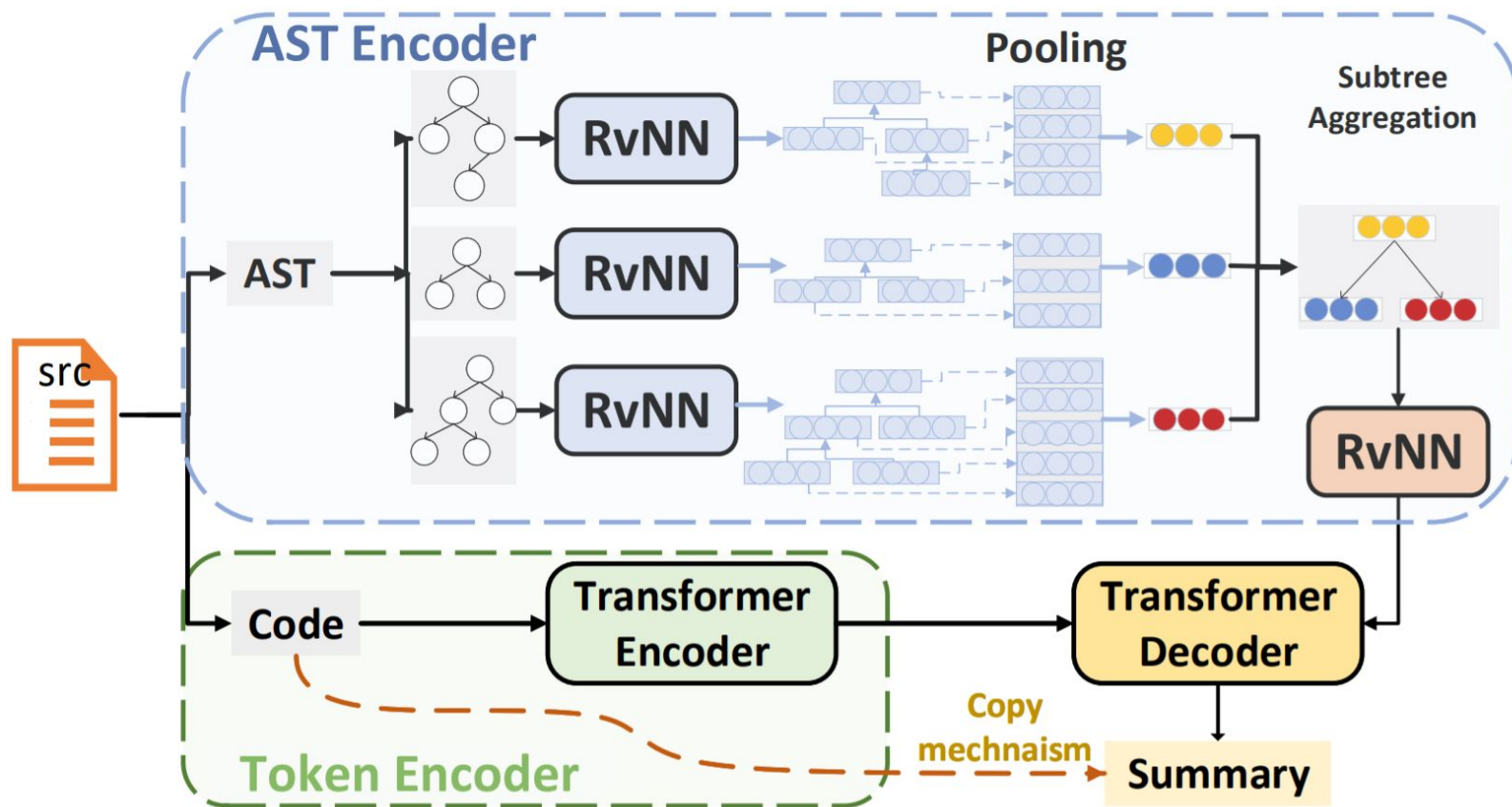
	Reference	loop through each of the columns in the given table, migrating each as a resource or relation
AST-based models	HybridDrl	finds a set of rows in a table
	Astattgru	maps a list to the table
	Hdeepcom	loop total columns of of column the the , , , column having . column
	Code2seq	returns the list of the given
	Astnn	returns the list of the given list of the given list .
	Attgru	executes the function , including the java attributes from all columns rules found
Others	NCS	migrate the columns of a table in the given resource query
	CodeNN	returns a foreign - of columns the

- AST를 tree 기반의 뉴럴넷으로 인코딩 → 학습 시간이 너무 길어짐
 - 높은 프로그램의 복잡도와 크기 때문
 - e.g. HybridDrl: AST 를 이진트리로 변형
⇒ 더욱더 깊은 트리를 만들게 되면서 정보 손실
- AST 직렬화 → AST 의 계층 정보의 손실
 - ASTNN: AST를 statement tree 로 분할 ⇒ 큰 tree 학습의 어려움을 해소
 - 하지만 각각의 서브 트리는 한 개의 statement 만을 포함할 수 있음
 - Sub-tree 각각을 직렬화하여 입력 ⇒ 계층 구조 정보의 손실

Solution: Hierarchical splitting and reconstruction

1. 전체 AST를 적절하게 분리
2. Tree 기반 모델로 각 sub-tree를 학습
3. 이후 조합
⇒ 전체 AST의 representation

Model Structure



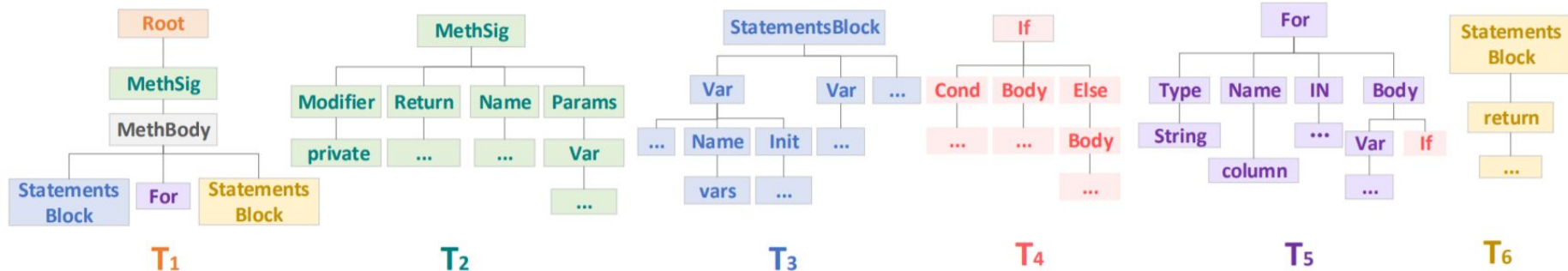
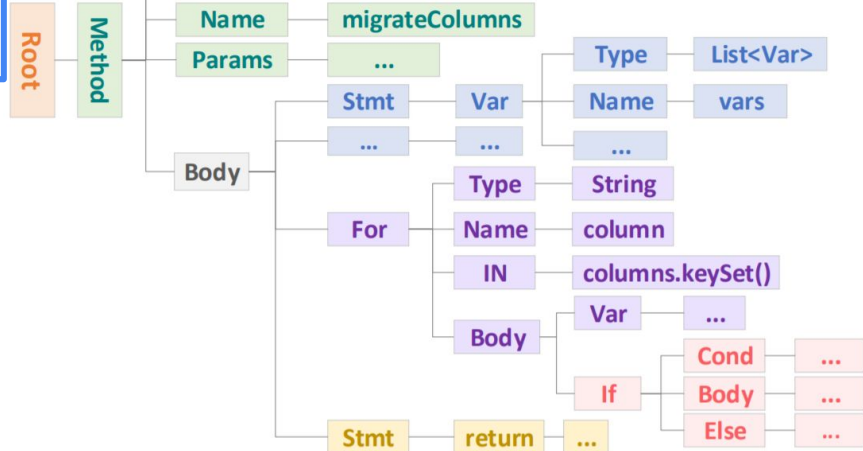
AST Encoder

- AST를 만들고 preorder 순회
- Composite structure (if, while) 방문
 - placeholder node 삽입
 - subtree 생성 $\rightarrow$ semantic을 placeholder로 배치

```

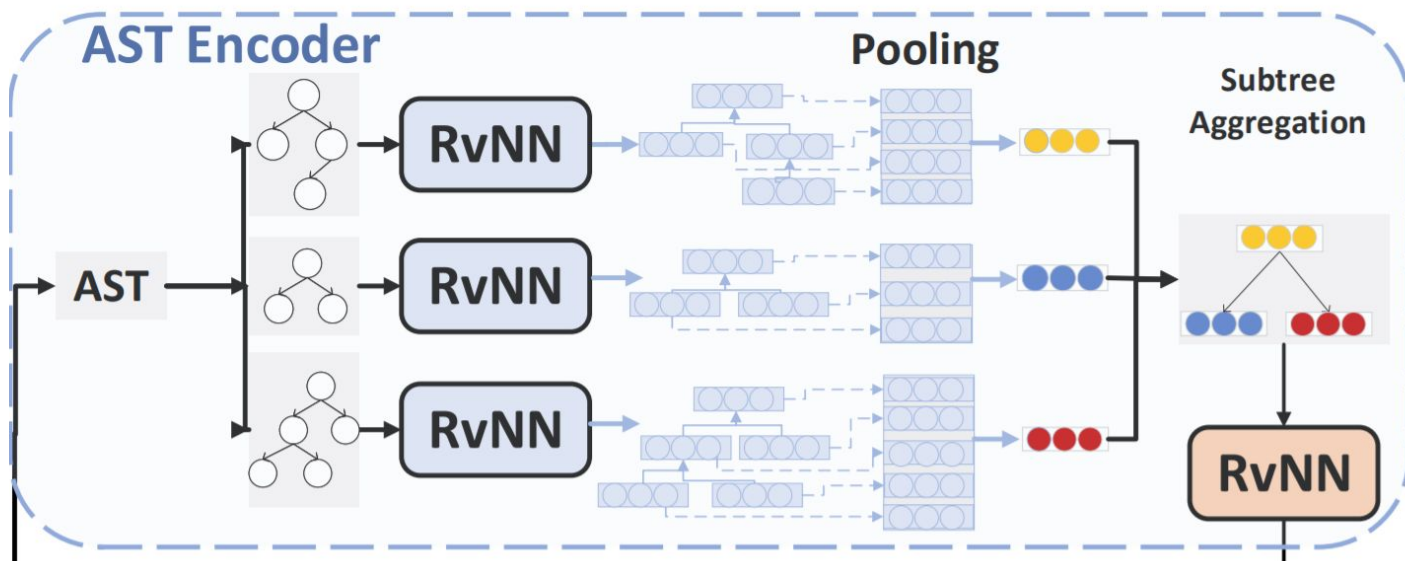
1. private Collection<Var> migrateColumns(SQLTable currentTable)
2. List<Var> vars=new ArrayList<>();
3. String tableType=currentTable.getEntityType();
4. Map<String,ResourceType.DataType> columns=currentTable.getColumns();
5. Map<String,String> foreignColumns=currentTable.getForeignKeyColumns();
6. for (String column : columns.keySet()) {
7.     ResourceType.DataType columnType=columns.get(column);
8.     if (foreignColumns.containsKey(column)) {
9.         vars.addAll(migrateAsRelation(tableType,
10.             column,foreignColumns.get(column));}
11.     else {
12.         vars.addAll(migrateAsResource(tableType,columnType,column));
13.     }
14. }
15. return vars;

```



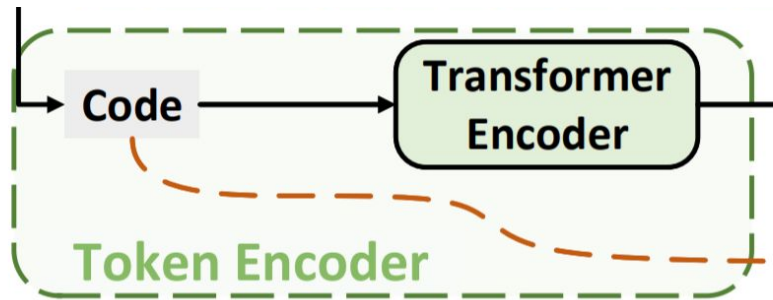
AST Encoding

1. Encode subtree: tree-based RvNN (Recursive NN) + max pooling
2. Hierarchical relationship: aggregation $\rightarrow$ RvNN



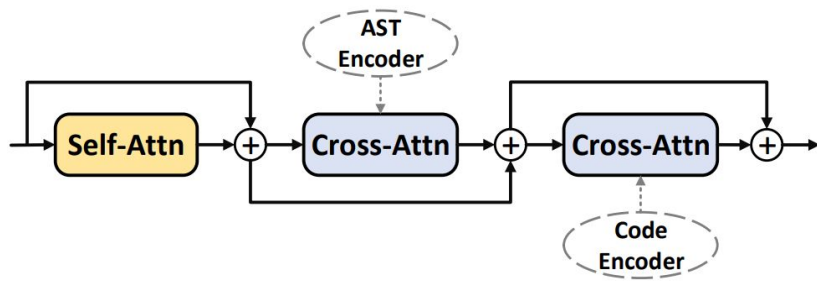
Code Token Encoder

- Transformer
 - multi-head self-attention + relative position embedding module
- Code token $\rightarrow$ output vector



Decoder w/ Copy Mechanism

- 2 Encoding source: AST encoder + code token encoder
⇒ Serial strategy



- Copy Mechanism: Input code에서 token을 복사
 - Attention layer로 copy probability 학습

Experiment - Setup

- Dataset (Java)
 - TL-CodeSum (83,661)
 - Funcom (2,111,230)
- Vocab size
 - AST: 10k
 - Code: 30k
 - Summary: 50k
- Metrics:
 - BLEU-CN ([0%, 100%])
 - Meteor ([0%, 100%])
 - Rouge-L ([0%, 100%])
 - Cider ([0, 10])

Result - Metrics

Model	Funcom				TL-CodeSum			
	BLEU	Met.	Rouge	Cider	BLEU	Met.	Rouge	Cider
CodeNN	20.93	11.44	29.09	0.90	22.22	14.08	33.14	1.67
HDeepcom	25.71	15.59	36.07	1.42	23.32	13.76	33.94	1.74
Attgru	27.82	18.10	39.20	1.84	29.72	17.03	38.49	2.35
NCS	29.18	19.94	40.09	2.15	40.63	24.85	52.00	3.47
Code2seq	23.84	13.84	33.65	1.31	16.09	8.94	24.21	0.66
HybridDrl	23.25	12.55	32.04	1.11	23.51	15.38	33.86	1.55
Astattgru	28.17	18.43	39.56	1.90	30.78	17.35	39.94	2.31
CodeAstnn	28.27	18.86	40.34	1.94	41.08	24.95	51.67	3.49
CAST _A	30.56	20.96	42.46	2.30	43.76	27.15	54.09	3.84
CAST _C	30.35	20.65	42.22	2.24	43.81	26.95	53.53	3.82
CAST	30.83	20.96	42.71	2.31	45.19	27.88	55.08	3.95

Result - Human Evaluation

Model	Informativeness	Naturalness	Similarity
CAST	2.74 (1.29)	3.08 (1.23)	2.66 (1.29)
Astattgru	2.26(1.05)	2.46(1.31)	2.02(1.09)
NCS	2.39(1.10)	2.78(1.19)	2.17(1.14)
CodeAstnn	2.44(1.08)	3.00(1.13)	2.20(1.14)

Thank You